

Student Management System

1.Spring Initializer

- Project :Gradle-Groovy,Language:Java,Springboot:3.5.3,Packaging :Jar,Java:21
- Dependencies:Spring web , Spring Data JPA ,PostgreSQL, Validation,Spring Boot Dev Tools

2.Postgre Cloud -Render

- Created a postgre instance in Render (Cloud)
- Host : dpg-d1n7pkeuk2gs739miamg-a
- Username : admin
- Password: iLrxrUitCzT6wxrpGI0Az0QdBLTOy4Ft
- Port: 5432
- Database URL: postgresql://admin:iLrxrUitCzT6wxrpGI0Az0QdBLTOy4Ft@dpg-d1n7pkeuk2gs739miamg-a.singapore-postgres.render.com/studentdb_h96v
- PSQL command : PGPASSWORD=iLrxrUitCzT6wxrpGI0Az0QdBLTOy4Ft psql -h dpg-d1n7pkeuk2gs739miamg-a.singapore-postgres.render.com -U admin studentdb_h96v
- Database : studentdb_h96v
- Since it is a Demo project I have used application.properties to connect with the database but in real time we use secured software for the connection.

3. Springboot Database Connection

- In application.properties I have added these credentials , so as connected the Database with Springboot API
- A success message is received when starting the springboot Application
- 2025-07-10T09:01:13.789+05:30 INFO 10836 --- [StudentManagementSystem] [restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2025-07-10T09:01:15.432+05:30 INFO 10836 --- [StudentManagementSystem] [restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@54743c1e
2025-07-10T09:01:15.434+05:30 INFO 10836 --- [StudentManagementSystem] [restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2025-07-10T09:01:15.598+05:30 WARN 10836 --- [StudentManagementSystem] [restartedMain] org.hibernate.orm.deprecation : HHH90000025: PostgreSQLDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2025-07-10T09:01:15.849+05:30 INFO 10836 --- [StudentManagementSystem] [restartedMain] org.hibernate.orm.connections.pooling : HHH10001005: Database info:

4. Local view of Cloud Database

- Single it is a cloud based instance in render to view it I have used a local desktop DBeaver 25.2.1
- Click Database→New Database connection → matching all the credentials
- The database along with its schema is visible in the local desktop

5. Postman

- As we are not using frontend end Points , Postman is used to talk to the Database using SpringBoot APIs
- We can use POST ,GET, PUT,DELETE with the urls to talk to the Springboot APIs
- My springboot Port is 8080 so the url maps to the controller using <http://localhost:8080/api/students>

6. Controller , Service , Entity ,Enums , Repository ,ServiceImpl .

- Controller talks to the Postman and get the JSON object from it & transfers to Service Layer
- Service Layer is where all the business logics can be coded and service layer calls the Repository which extends JPA repository to fetch data from the database .
- Repository takes the student details as per its requirements either using Query or by the service layer accessibilities .
- Get ,Put ,Delete, Update ,Post functionalities are done as per the requirements
- The complexity –Pagination, Sorting , Filtering are done using Pageable and Specification interfaces in Spring Data JPA ,used to achieve the Pagination ,Sorting & Filtering.

7.Postman URL and JSON objects :

- POST – <http://localhost:8080/api/students>

```
{
  "firstname": "Poojithanima",
  "lastname": "Svan",
  "email": "pima.selvan@example.com",
  "dateOfBirth": "10-07-2000",
  "enrollmentDate": "01-06-2024",
  "gpa": 8.5
}
```
- GET- <http://localhost:8080/api/students/1>
- DELETE- <http://localhost:8080/api/students/1>
- PUT-<http://localhost:8080/api/students/1>

```
{
  "firstname": "Raj",
  "gpa": 8.4
}
```

Pagination , Filtering ,Sorting :

- GET- <http://localhost:8080/api/students?page=0&size=5>
- GET- <http://localhost:8080/api/students?status=ACTIVE&minGpa=5.0&maxGpa=9.0>
- GET- <http://localhost:8080/api/students?page=0&size=5>
- GET- <http://localhost:8080/api/students?sort=gpa,desc>
- GET- <http://localhost:8080/api/students?sort=firstName,asc&sort=gpa,desc>
- GET- <http://localhost:8080/api/students?status=ACTIVE>
- GET- <http://localhost:8080/api/students?status=ACTIVE&minGpa=6.0&maxGpa=9.0&keyword=b&page=0&size=5&sort=firstName,asc&sort=gpa,desc>

Separate Filter Logics in Controller :

- Three separate PUT Mappings for filtering using status, gpa,Name are also implemented

- Filtering Status -PUT- <http://localhost:8080/api/students/filter/status?status=ACTIVE>
- Filtering gpa-PUT- <http://localhost:8080/api/students/filter/gpa?minGpa=6.0&maxGpa=9.5>
- Filtering Name -PUT- <http://localhost:8080/api/students/filter/Name?name=poor>

Error Validations :

- For Email Validation

POST – <http://localhost:8080/api/students>

```
{
  "firstname": "Poornima",
  "lastname": "Selvan",
  "email": "poora.selvanexample.com",
  "dateOfBirth": "10-07-2000",
  "enrollmentDate": "01-06-2024",
  "gpa": 8.5
}
```

- 404 Error

GET- <http://localhost:8080/api/students/9>